

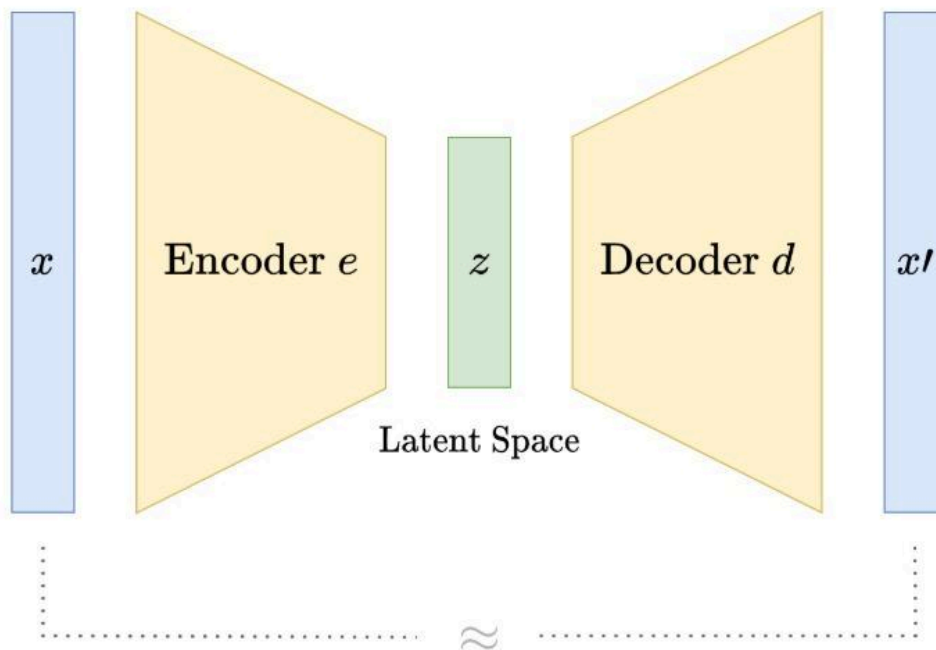
Autoencoder and Variation autoencoder

1.define auto conder or draw architecture of autoconder and execute for simple application on your own

An Autoencoder is a type of neural network used for unsupervised learning. It learns to compress data (encoding) into a smaller representation and then reconstruct it (decoding) back to the original form.

- Encoder that captures important features by reducing dimensionality
- Decoder that rebuilds the data from this compressed representation

Architecture of Autoencoder



An Autoencoder consists of three main parts:

1. Encoder

The encoder compresses the input data into a lower-dimensional representation.

- **Input Layer:** Receives original data (image, text, numbers).
- **Hidden Layers:** Apply weights and activation functions to reduce dimensionality and extract important features.
- **Latent Output:** Produces a compressed vector called the latent representation.

2. Bottleneck (Latent Space)

- Smallest layer of the network.
- Stores compressed representation.
- Forces the model to retain only important features.

3. Decoder

Reconstructs the original data from the latent representation.

- **Hidden Layers:** Expand compressed data.
- **Output Layer:** Produces reconstructed data similar to input.

Loss Function

Autoencoder minimizes reconstruction loss.

Common loss functions:

- **Mean Squared Error (MSE)** – for continuous data
- **Binary Cross Entropy (BCE)** – for binary data

Training is done using backpropagation to reduce reconstruction error.

Simple Application on autoencoder

```
# =====  
  
# 1. Import Libraries  
  
# =====  
  
import numpy as np  
  
import matplotlib.pyplot as plt  
  
import tensorflow as tf  
  
from tensorflow.keras import layers, models  
  
from tensorflow.keras.datasets import mnist  
  
# =====  
  
# 2. Load Dataset  
  
# =====  
  
(x_train, _), (x_test, _) = mnist.load_data()  
  
# Normalize images (0 to 1)  
  
x_train = x_train.astype("float32") / 255.  
x_test = x_test.astype("float32") / 255.  
  
# Reshape to (samples, 784)  
  
x_train = x_train.reshape((len(x_train), 784))  
x_test = x_test.reshape((len(x_test), 784))  
  
# =====  
  
# 3. Add Noise  
  
# =====
```

```

noise_factor = 0.5

x_train_noisy = x_train + noise_factor *
np.random.normal(size=x_train.shape)

x_test_noisy = x_test + noise_factor * np.random.normal(size=x_test.shape)

# Clip values between 0 and 1

x_train_noisy = np.clip(x_train_noisy, 0., 1.)

x_test_noisy = np.clip(x_test_noisy, 0., 1.)

# =====

# 4. Build Autoencoder

# =====

# Encoder

input_img = layers.Input(shape=(784,))

encoded = layers.Dense(128, activation='relu')(input_img)

encoded = layers.Dense(64, activation='relu')(encoded)

encoded = layers.Dense(32, activation='relu')(encoded)

# Decoder

decoded = layers.Dense(64, activation='relu')(encoded)

decoded = layers.Dense(128, activation='relu')(decoded)

decoded = layers.Dense(784, activation='sigmoid')(decoded)

# Combine

autoencoder = models.Model(input_img, decoded)

# Compile

autoencoder.compile(optimizer='adam', loss='binary_crossentropy')

```

```

autoencoder.summary()

# =====

# 5. Train Model

# =====

autoencoder.fit(

    x_train_noisy, x_train,

    epochs=10,

    batch_size=256,

    shuffle=True,

    validation_data=(x_test_noisy, x_test)

)

# =====

# 6. Test & Visualize Results

# =====

decoded_imgs = autoencoder.predict(x_test_noisy)

# Display results

n = 10

plt.figure(figsize=(20, 6))

for i in range(n):

    # Noisy image

    ax = plt.subplot(3, n, i + 1)

    plt.imshow(x_test_noisy[i].reshape(28, 28), cmap='gray')

    plt.title("Noisy")

```

```

plt.axis("off")

# Clean image

ax = plt.subplot(3, n, i + 1 + n)

plt.imshow(x_test[i].reshape(28, 28), cmap='gray')

plt.title("Original")

plt.axis("off")

# Reconstructed image

ax = plt.subplot(3, n, i + 1 + 2*n)

plt.imshow(decoded_imgs[i].reshape(28, 28), cmap='gray')

plt.title("Denoised")

plt.axis("off")

plt.show()

```

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 32)	2,080
dense_3 (Dense)	(None, 64)	2,112
dense_4 (Dense)	(None, 128)	8,320
dense_5 (Dense)	(None, 784)	101,136

Total params: 222,384 (868.69 KB)

Trainable params: 222,384 (868.69 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/10

235/235 ————— 6s 18ms/step - loss: 0.3366 - val_loss: 0.2107

Epoch 2/10

235/235 ————— 4s 19ms/step - loss: 0.1972 - val_loss: 0.1717

Epoch 3/10

235/235 ————— 4s 18ms/step - loss: 0.1694 - val_loss: 0.1607

Epoch 4/10

235/235 ————— 4s 17ms/step - loss: 0.1582 - val_loss: 0.1516

Epoch 5/10

235/235 ————— 5s 19ms/step - loss: 0.1510 - val_loss: 0.1462

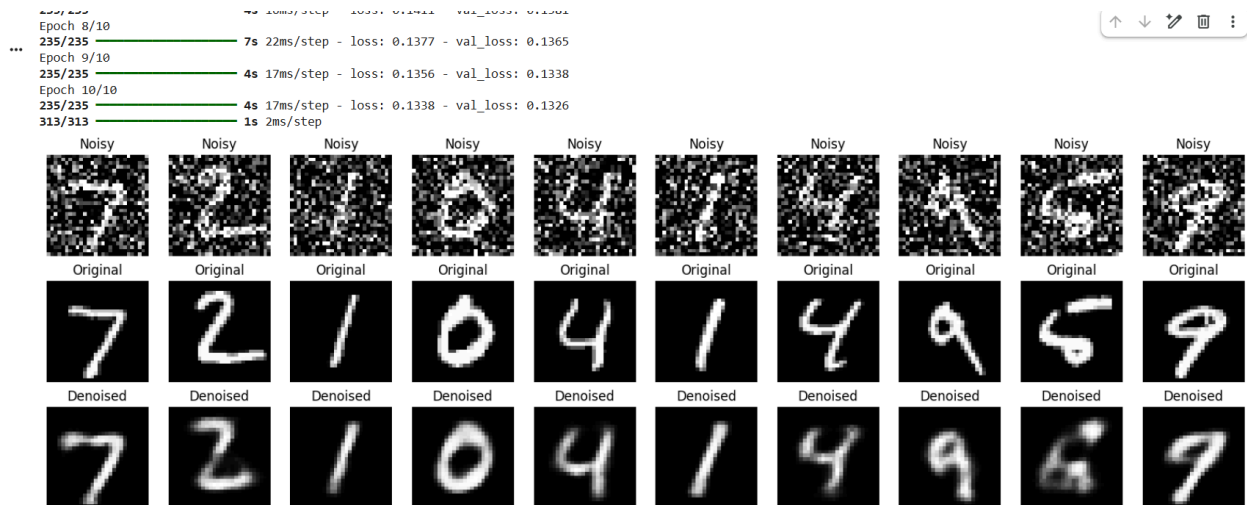
Epoch 6/10

235/235 ————— 4s 18ms/step - loss: 0.1454 - val_loss: 0.1408

Epoch 7/10

235/235 ————— 4s 16ms/step - loss: 0.1411 - val_loss: 0.1381

Epoch 8/10



2. what is variational autoencoder variation autoencoder with a simple example of ur own

A **Variational Autoencoder (VAE)** is a probabilistic version of an autoencoder.

Instead of learning a fixed compressed vector, it learns the **probability distribution (mean and variance)** of the latent space.

It can both:

- Reconstruct input data
- Generate new data samples

A VAE consists of three main components:

1. Encoder
2. Latent Space (Probabilistic Representation)
3. Decoder

2.1 Encoder

The encoder takes input data and converts it into two vectors:

- Mean (μ)
- Variance (σ^2)

Instead of directly producing a single latent vector, the encoder predicts the parameters of a probability distribution.

2.2 Latent Space

The latent space represents a probability distribution rather than a fixed value.

The latent variable is sampled using the reparameterization trick:

$$z = \mu + \sigma \cdot \varepsilon$$

Where ε is random noise sampled from a standard normal distribution.

This ensures the model remains differentiable and trainable using backpropagation.

2.3 Decoder

The decoder takes the sampled latent vector (z) and reconstructs the input data.

It attempts to generate output that closely resembles the original input.

3. Loss Function in VAE

The loss function of VAE consists of two parts:

1. Reconstruction Loss
 - Measures difference between original input and reconstructed output.

- Commonly uses Mean Squared Error (MSE) or Binary Cross Entropy (BCE).

2. KL Divergence Loss

- Ensures that the learned latent distribution is close to a standard normal distribution.
- Encourages smooth and continuous latent space.

Total Loss = Reconstruction Loss + KL Divergence

4. Simple Example (Own Example)

Consider a dataset of handwritten digits. When digit “3” is given as input:

The encoder outputs:

Mean (μ) = 0.5

Variance (σ^2) = 0.1

Different samples of z are generated from this distribution.

The decoder then produces:

- Slightly thick “3”
- Slightly tilted “3”
- Slightly curved “3”

Thus, the VAE can generate new variations of digits that were not explicitly present in the training dataset.

3. compare autoconder with variation autoencoder

Feature	Autoencoder	Variational Autoencoder
Type	Deterministic	Probabilistic
Latent Space	Fixed vector	Distribution (μ , σ^2)
Loss Function	Reconstruction loss only	Reconstruction + KL Divergence
Data Generation	Limited	Can generate new data
Complexity	Simple	More complex
Main Purpose	Feature learning	Generative modeling